CHAPTER 3
METHODOLOGY AND SYSTEM ARCHITECTURE

3.1 Introduction

Autonomous navigation of Unmanned Aerial Vehicles (UAVs) in urban environments without continuous Global Navigation Satellite System (GNSS) aiding represents one of the most critical frontiers in modern robotics, estimation theory, and intelligent flight control. In metropolitan environments, direct satellite signals are frequently degraded, severely attenuated, or entirely eliminated by tall architectural structures—a phenomenon universally known as the 'urban canyon' effect. Furthermore, tactical and operational scenarios increasingly require UAVs to execute low-altitude missions beneath bridges, within industrial warehouses, through indoor-outdoor transition corridors, and under adverse meteorological disturbances (such as heavy precipitation, wind gusts, and thermal turbulence), where GNSS positioning is either physically unavailable or vulnerable to electromagnetic jamming and spoofing.

Under unassisted inertial navigation, a quadrotor relies exclusively on onboard strapdown Inertial Measurement Units (IMUs) comprising tri-axial gyroscopes and tri-axial accelerometers. When integrated over time, sensor noise, stochastic thermal drift, and unmodeled bias instabilities cause the estimated kinematic state to diverge quadratically with time in position and linearly in velocity. Classical Error-State Extended Kalman Filters (ESKF) running in prediction-only mode can propagate uncertainty covariances rigorously, but they fundamentally cannot eliminate unobserved bias or arrest dead-reckoning drift without external aiding measurements.

To surmount this fundamental limitation, this graduation project investigates an innovative hybrid methodology: harnessing deep recurrent neural networks (specifically Gated Recurrent Units, GRU) to learn nonlinear motion-drift corrections directly from temporal kinematic patterns, sensor uncertainty, and attitude representations. When treated as an artificial velocity pseudo-measurement and strictly regulated by rigorous statistical safety gates, this learned correction can recursively arrest inertial drift in real time. This chapter presents an exhaustive,

technically rigorous account of the end-to-end methodology: the curated 1.422-million-sample multi-source real-flight corpus, the 15-state ESKF kinematic formulation, the GRU v3 temporal architecture, the on-policy training regime, the multi-tiered safety gating policy, the integration into a real-time Three.js 3D urban simulation, and empirical trajectory evaluations conducted across multiple flight geometries and weather conditions.

3.2 Overall System Architecture

The proposed system is structured around an asynchronous, multi-rate, decoupled client-server architecture designed to resolve the computational tension between real-time 60–100 Hz physical simulation and computationally intensive deep learning inference. In monolithic simulation frameworks, executing deep neural networks directly inside the physics or rendering loop triggers thread blocking, dropped frames, and unstable numerical integration of quadrotor dynamics. Our architecture enforces strict modular separation across three distinct computational tiers interconnected via high-speed local TCP/IP networking, as illustrated in Figure 3.1.

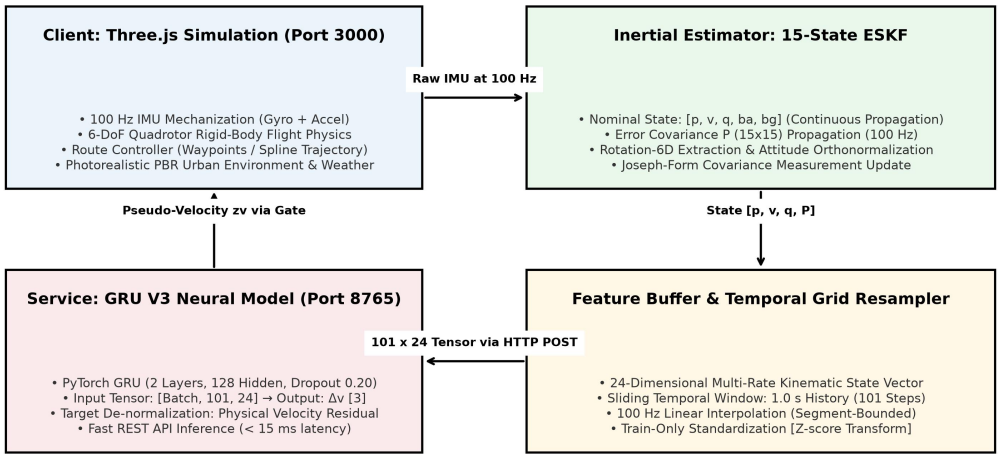


Figure 3.1: Complete Multi-Tier System Architecture and Inter-Service Communication Dataflow

The complete system comprises three tightly synchronized subsystems operating across dedicated ports:

- **Tier 1 — High-Fidelity WebGL Simulation Client (Port 3000):** Built using Three.js and bundled with Vite. This client executes the 6-DoF quadrotor rigid-body equations of motion at a fixed 100 Hz simulation timestep, generates procedural 3D urban environments with Physically Based Rendering (PBR) materials, simulates synthetic sensor noise (IMU, barometer, magnetometer), and hosts the user-interactive route-drawing interface.

- **Tier 2 — Python FastAPI GRU V3 Inference Bridge (Port 8765):** Hosts the frozen PyTorch GRU V3 deep neural network (`PROJECT_GHOST_GRU_V3_BEST_VALIDATION.pt`). It receives batches of 101x24 temporal feature tensors via HTTP POST requests, executes GPU/CPU forward inference, de-normalizes the predictions, and returns the physical 3D velocity residual vector delta_v in under 15 milliseconds.
- **Tier 3 — Python FastAPI PULP-DroNet Vision Avoidance Bridge (Port 8766):** Hosts the lightweight convolutional ResBlock-1.0 vision model. It processes 200x200 grayscale sensor frames rendered from the quadrotor's forward-facing virtual camera, predicting steering offset angles and collision probabilities to provide reactive obstacle avoidance overrides.

A foundational architectural principle implemented in this project is the Dual-State ESKF Formulation (C0 Architecture). To avoid corrupting the neural network's input feature distribution through uncontrolled recursive feedback, two distinct estimator streams are maintained simultaneously: (1) The Feature ESKF, which runs in pure prediction-only mode using raw IMU integration, precisely preserving the nominal state distribution on which the neural network was trained; and (2) The Aided ESKF, which incorporates the statistically gated pseudo-velocity corrections to maintain the accurate state estimate utilized by the flight controller and HUD. Table 3.1 summarizes the operational specifications and communication contracts of each system module.

Table 3.1: System Modules, Network Ports, Execution Rates, and Operational Contracts

System Module	Network Endpoint	Execution Rate	Data Payload / Tensor Contract	Target Latency
Three.js Simulation	http://localhost:3000	100 Hz (Physics) 60 FPS (Render)	6-DoF Rigid Body States, IMU Synthesizer, 3D Mesh	≤ 16.6 ms
Inertial ESKF Engine	Internal Client Loop	100 Hz Fixed Step	15-State Error Vector, 15x15 Covariance Propagation	≤ 1.2 ms
Feature Grid Resampler	Internal Buffer Loop	1.0 s Window / 3 s Cadence	Rolling Buffer → 101x24 Interpolated Float Tensor	≤ 3.5 ms
GRU V3 Inference Server	http://127.0.0.1:8765/infer	On-Demand (0.33–2.0 Hz)	Input: [1, 101, 24] → Output: delta_v [3] (m/s)	≤ 15.0 ms

3.3 Development Environment and Tools

To guarantee experimental reproducibility and rigorous engineering standards, all software frameworks, numerical packages, and runtime environments were cataloged and locked. Table 3.2 details the complete development environment utilized throughout the design, offline training, and real-time execution phases.

Table 3.2: Software Tools, Runtime Environments, Framework Versions, and System Roles

Tool / Library	Version	Ecosystem / Environment	Specific Role in System Implementation
Python	3.11.2 (64-bit)	Conda / Virtualenv (.venv)	Core backend language for model training, data pipelines, and REST APIs
PyTorch	2.2.0 + CUDA 12.1	Python Runtime	Deep learning framework for GRU V3 and DroNet neural execution
FastAPI	0.115.0	Python Web Server	Asynchronous high-performance RESTful API microservice bridge
Uvicorn	0.30.0 (Standard)	ASGI Server Worker	Lightweight HTTP server hosting model endpoints on ports 8765 and 8766
NumPy & SciPy	1.26.4 / 1.13.0	Scientific Python	Numerical matrix linear algebra, quaternion kinematics, and covariance math
Three.js	0.170.0 (r170)	Node.js / Browser WebGL	Hardware-accelerated 3D city rendering, camera rigging, and flight visuals
Vite	6.0.0	Node.js Build Tool	Fast HMR development server serving the simulation frontend on port 3000
Matplotlib	3.11.2	Data Visualization	Scientific rendering of training curves, error distributions, and trajectory charts

The physical host workstation was equipped with an Intel Core i7 processor, 32 GB of system DDR4 RAM, and an NVIDIA GeForce RTX GPU with CUDA Compute Capability 8.9. All Python microservices utilize dedicated virtual environments (`.venv`), ensuring zero dependency contamination across system modules.

3.4 Dataset Preparation and Data Integration

A foundational strength of this research is its grounding in a massive, multi-source, real-flight empirical dataset rather than idealized mathematical toy problems. The final curated research database comprises exactly 1,422,348 synchronized feature rows collected across 19 complete drone flight recordings spanning three internationally recognized UAV research corpora: INSANE, LandingUAV, and PanoAir. Figure 3.2 illustrates the relative distribution of samples across the three source domains.

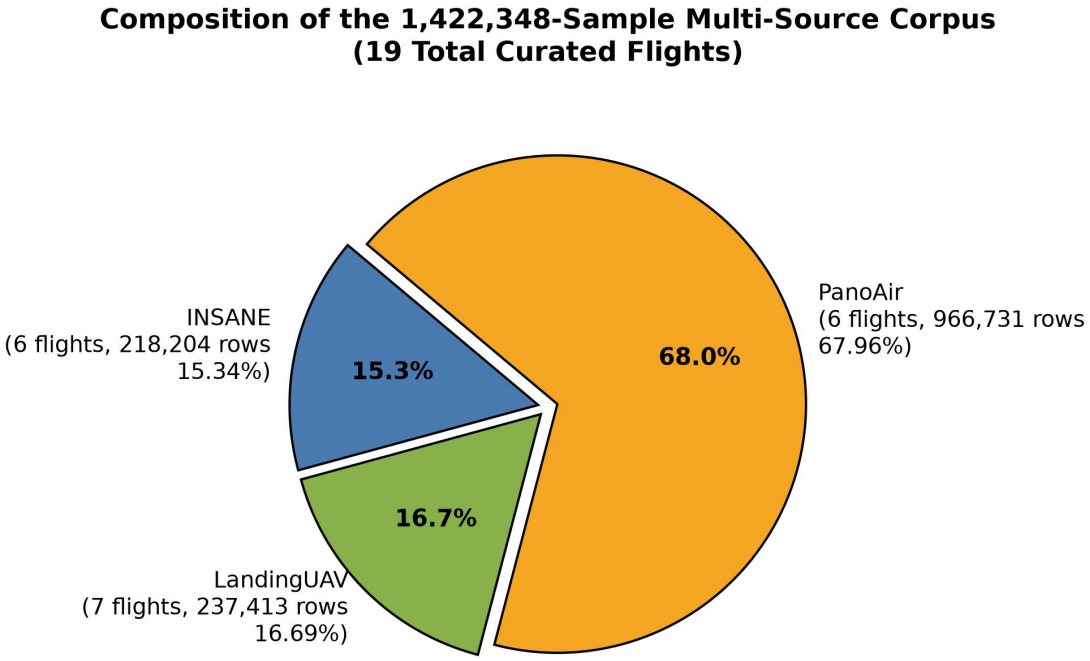


Figure 3.2: Composition of the 1,422,348 Real-Data Feature Corpus Across 19 Curated Flights

3.4.1 Multi-Source Corpus Characteristics and Operational Regimes

Each constituent dataset contributes distinct vehicle dynamics, sensor grades, sampling rates, and flight environments:

- **INSANE Dataset (6 Flights, 218,204 Rows, 15.34%):** Captured across challenging analog environments, including Mars-analog desert terrains, vertical rocky cliffs, canyon corridors, and complex outdoor-to-indoor transition flights. The raw data exhibited severe IMU channel provenance issues in early versions; an exhaustive PX4 native log rebuild was executed (Notebook L02) to extract authentic raw accelerometer and gyro channels without spurious interpolation.
- **LandingUAV Dataset (7 Flights, 237,413 Rows, 16.69%):** Captured in maritime and northern European settings (Hamburg boathouse, maritime lock facilities, and Wicklow

hills, Ireland). Crucially, these recordings feature environmental metadata tags representing adverse weather, specifically light rain, continuous precipitation, and high-velocity wind gusts.

- **PanoAir Dataset (6 Flights, 966,731 Rows, 67.96%):** Represents high-frequency urban and perimeter flights executed under varying solar illumination profiles, including direct sunlight, overcast cloud cover, and night-time operation. PanoAir provides long-duration continuous trajectories essential for evaluating extended temporal drift.

3.4.2 Environmental Metadata Stratification vs. Physical Telemetry

A vital academic distinction enforced throughout this research (Notebook L01 and L04) concerns the interpretation of weather tags. The curated dataset contains categorical metadata descriptors such as 'Light Rain + Wind Gusts', 'Rain / Calm Wind / Maritime', 'Night', and 'Mars Analog Desert'. While these tags provide valuable domain stratification for evaluating model robustness, they do not represent synchronized numerical meteorological telemetry (e.g., anemometer wind speed in m/s or rain gauge precipitation rate in mm/h). Consequently, performance comparisons across weather conditions are descriptive regime evaluations rather than causal wind-tunnel ablations. Table 3.3 catalogs the complete 19-flight ledger, split assignments, environmental tags, and row counts.

Table 3.3: Master 19-Flight Research Ledger: Sources, Environmental Tags, Splits, and Samples

Flight ID	Source Dataset	Assigned Split	Environmental Regime Tag	Feature Rows
mars_11	INSANE	TRAIN	Mars analog desert terrain	29,980
mars_12	INSANE	TRAIN	Mars analog / cliff corridor	18,125
mars_14	INSANE	TRAIN	Mars analog / canyon corridor	34,522
outdoor_1	INSANE	TRAIN	Open outdoor flight corridor	38,755
hamburgBoathouse_1	LandingUAV	TRAIN	Rain / calm wind / maritime	16,160
hamburgLock_2	LandingUAV	TRAIN	Rain / calm wind / maritime	10,720
wicklowTest_4	LandingUAV	TRAIN	Light rain / wind gusts	39,506
wicklowTest_7	LandingUAV	TRAIN	Light rain / wind gusts	61,210
wicklowTest_8	LandingUAV	TRAIN	Light rain / wind gusts	38,465

seq1	PanoAir	TRAIN	Overcast / cloudy sky	78,621
seq17	PanoAir	TRAIN	Night-time low illumination	228,501
seq5	PanoAir	TRAIN	Cloudy / scattered overcast	152,272
seq6	PanoAir	TRAIN	Sunny / clear atmospheric	177,139
indoor_1	INSANE	VALIDATION	Enclosed indoor warehouse	61,323
wicklowTest_5	LandingUAV	VALIDATION	Light rain / wind gusts	58,236
seq10	PanoAir	VALIDATION	Night-time operational	121,418
transition_3	INSANE	TEST (Held-out)	Outdoor-to-indoor transition	35,499
hamburgLock_3	LandingUAV	TEST (Held-out)	Rain / calm wind / maritime	13,116
seq9	PanoAir	TEST (Held-out)	Sunny / clear daylight	208,780

3.4.3 Rigorous Whole-Flight Data Partitioning Protocol

In sequential time-series modeling, randomly shuffling overlapping temporal windows into train and test sets constitutes severe methodological data leakage, as adjacent windows share over 95% of their raw inertial signal history. To enforce absolute integrity, our research utilizes a Whole-Flight Partitioning Protocol: the unit of independence is an entire flight mission. Thirteen flights (923,976 rows) form the Training Partition; three flights (240,977 rows) form the Validation Partition; and three flights (257,395 rows) are retained as an untouched Held-Out Test Partition. Normalization statistics and gating thresholds were fitted exclusively on training flights.

3.5 Machine-Learning Model Architecture

The core neural estimation component is Project Ghost GRU V3, a deep recurrent neural network designed to map high-frequency temporal IMU dynamics and estimator uncertainties into 3D velocity residual vectors. This section outlines the feature vector formulation, mathematical gating equations, and network layer topology.

3.5.1 The 24-Dimensional Input Feature Vector

At each time step, the model ingests a precisely structured 24-dimensional feature vector. Ground-truth coordinates, absolute positions, flight IDs, and timestamps are strictly excluded to prevent information leakage. Table 3.4 details the complete feature ordering, physical units, and engineering rationale.

Table 3.4: The 24-Dimensional Input Feature Vector Definition and Channel Order

Feature Indices	Channel Category	Channel Identifiers	Physical Units & Mathematical Description
0 – 2	Raw Angular Rates	feat_gx, feat_gy, feat_gz	rad/s — Tri-axial body-frame gyroscope angular velocities
3 – 5	Raw Specific Force	feat_ax, feat_ay, feat_az	m/s^2 — Tri-axial body-frame accelerometer linear specific force
6 – 8	ESKF Predicted Velocity	feat_eskf_vx, vy, vz	m/s — World-frame velocity components from inertial propagation
9 – 14	Rotation-6D Attitude	r11, r21, r31, r12, r22, r32	Unitless — Continuous 6D representation (first two columns of R_WB)
15 – 17	Position Uncertainty	sigma_px, sigma_py, sigma_pz	m — Square root of diagonal position error covariance blocks in P
18 – 20	Velocity Uncertainty	sigma_vx, sigma_vy, sigma_vz	m/s — Square root of diagonal velocity error covariance blocks in P
21 – 23	Attitude Uncertainty	sigma_th_x, th_y, th_z	rad — Square root of diagonal small-angle attitude error covariance in P

A critical engineering design choice is the Rotation-6D representation (Zhou et al., 2019). Euler angles suffer from kinematic singularities (gimbal lock), while 4D unit quaternions suffer from antipodal double-cover ambiguity (q and -q represent the same rotation), which severely disrupts neural gradient descent. By taking the first two column vectors of the body-to-world rotation matrix R_WB, we obtain an unconstrained 6D continuous representation that is uniquely differentiable and orthonormalized via Gram-Schmidt at runtime.

3.5.2 Mathematical Formulation of Gated Recurrent Units (GRU)

Traditional Recurrent Neural Networks (RNNs) suffer from vanishing and exploding gradients over long temporal horizons. While Long Short-Term Memory (LSTM) networks

resolve this using three gating mechanisms (input, forget, and output gates), the Gated Recurrent Unit (Cho et al., 2014) couples the input and forget mechanisms into a single update gate, reducing the parameter count by approximately 25% while maintaining equivalent representational capacity. For an input vector x_t in R^24 and prior hidden state h_{t-1} in R^128, the recurrence equations at timestep t are formulated as:

$$r_t = \sigma(W_r \cdot x_t + U_r \cdot h_{t-1} + b_r) \quad \textbf{(Equation 3.1: Reset Gate)}$$

$$z_t = \sigma(W_z \cdot x_t + U_z \cdot h_{t-1} + b_z) \quad \textbf{(Equation 3.2: Update Gate)}$$

$$\tilde{h}_t = \tanh(W_h \cdot x_t + U_h \cdot (r_t \odot h_{t-1}) + b_h) \quad \textbf{(Equation 3.3: Candidate State)}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad \textbf{(Equation 3.4: Hidden State Update)}$$

where sigma(·) denotes the logistic sigmoid activation function, tanh(·) represents the hyperbolic tangent, and circle-dot denotes element-wise Hadamard multiplication. The reset gate r_t determines how much past state history to discard, while the update gate z_t controls the balance between retaining the previous hidden state h_{t-1} and adopting the candidate state $\tilde{h}_t$.

3.5.3 Complete Network Topology and Projection Head

The complete PyTorch architecture consists of an input dimension of 24, two stacked recurrent GRU layers with hidden dimension 128 and inter-layer dropout of 0.20, followed by Layer Normalization (LayerNorm(128)) applied to the final sequence timestep h_101. The normalized hidden state is projected through a multi-layer perceptron head: Linear(128 -> 64) -> ReLU -> Dropout(0.10) -> Linear(64 -> 3). The output vector represents the normalized 3D velocity residual pred_norm in R^3. Total trainable parameters equal exactly 166,915 weights.

3.6 Model Training Methodology

The model training program progressed through three distinct research iterations: (1) baseline architecture comparison, (2) discovery and diagnosis of closed-loop distribution shift, and (3) on-policy training with dataset augmentation.

3.6.1 Preliminary Supervised Architecture Selection: LSTM vs. GRU

In the initial exploratory phase (Notebook L07), identical supervised window-level prediction tasks were executed to compare an LSTM architecture against a GRU architecture. As documented in the project logs, the GRU achieved a validation macro velocity RMSE of 41.952 m/s (best epoch 15) compared to 71.631 m/s for the LSTM (best epoch 7), while utilizing 24%

fewer parameters (166,915 vs. 219,651). Consequently, the GRU was selected as the foundational model.

3.6.2 The Closed-Loop Failure of Early C0 Integration

Crucially, when the early supervised GRU model was integrated into a recursive closed-loop Kalman filter (the initial C0 experiment, Notebook L08), the results were catastrophic on held-out tests. On the transition_3 flight, position RMSE increased from 20,040 m to 41,209 m (-105.63% degradation); on hamburgLock_3, error increased from 91.5 m to 131.9 m (-44.22%); and on seq9, error increased from 4,595 m to 6,969 m (-51.66%). Three-flight macro position RMSE doubled from 8,242 m to 16,103 m.

This failure revealed a critical scientific lesson: Closed-Loop Covariate Shift. In supervised training, the network only saw features generated along the prediction-only ESKF drift manifold. In a recursive loop, each applied velocity update slightly alters subsequent state estimates and uncertainties. If the network outputs even a minor erroneous prediction, the estimator enters an unfamiliar feature regime, causing the recurrent model to output larger erroneous residuals, rapidly triggering positive feedback and catastrophic divergence.

3.6.3 Iteration 3: On-Policy Training with Balanced Multi-Regime Augmentation

To resolve closed-loop shift, the training methodology was completely overhauled in Iteration 3 (Notebook L09). Rather than training only on open-loop traces, training windows were generated from three balanced behavioral regimes: (1) 2,771 Open-Loop windows from pure prediction-only traces; (2) 2,771 V1 Closed-Loop windows from historical integration rollouts; and (3) 2,771 V2 On-Policy windows visited by the improved aided estimator with newly re-matched residual targets. This yielded 8,313 total training windows.

Training was executed using the Adam optimizer with initial learning rate 1e-3, batch size 64, Mean Squared Error (MSE) loss, and early stopping. Training converged at epoch 30 (checkpoint `PROJECT_GHOST_GRU_V3_BEST_VALIDATION.pt`), achieving a validation macro velocity RMSE of 2.2397 m/s, global RMSE of 2.3259 m/s, and MAE of 1.7475 m/s. Table 3.5 summarizes the training parameters.

Table 3.5: Final GRU V3 Training Hyperparameters, Dataset Regimes, and Optimization Settings

Hyperparameter / Training Setting	Configured Value	Methodological Rationale
Total Training	8,313 Augmented	Balanced across 2,771 Open-Loop + 2,771 V1

Windows	Windows	Closed + 2,771 V2 On-Policy
Context Window Length	1.0 second (101 timesteps at 100 Hz)	Captures complete quadrotor dynamic transient history with endpoints
Optimizer & Learning Rate	Adam, Initial lr = 1.0e-3	Adaptive moment estimation with ReduceLROnPlateau decay
Loss Function	Mean Squared Error (MSE)	Penalizes Euclidean velocity residual errors equally across axes
Batch Size & Epochs	Batch Size = 64, Max Epochs = 44	Best validation checkpoint selected at Epoch 30
Checkpoint Metric	Validation Macro Velocity RMSE	2.2397 m/s on V2 on-policy validation rollouts
Model Checkpoint Hash	SHA-256: f4158ff782b7...	Verified cryptographic hash locked in configuration manifest

3.7 Integration of the Trained Models into the Complete System

Deploying deep neural networks within recursive estimation pipelines requires bulletproof safety mechanisms. Unbounded neural outputs can corrupt Kalman filter state covariance, causing filter divergence. This project implements a Four-Stage Cascaded Safety Gate to validate every incoming prediction before applying it to the estimator, as illustrated in Figure 3.3.

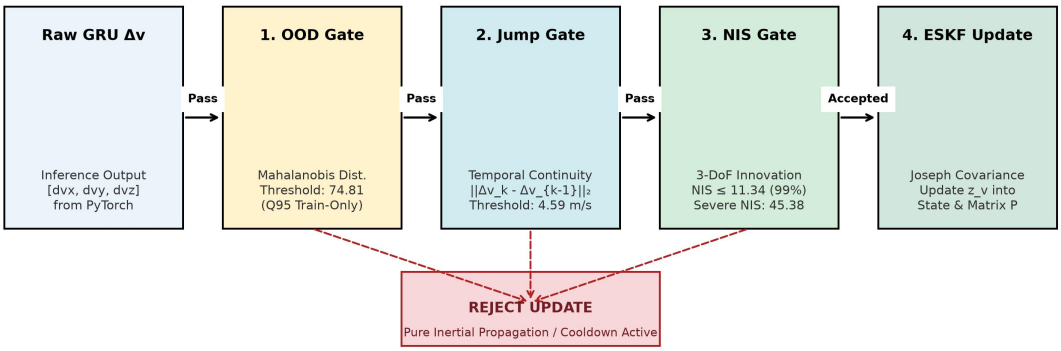


Figure 3.3: The Four-Stage Cascaded Safety Gating Pipeline for Guarded Kalman Updates

3.7.1 Four-Stage Safety Gating Formulation

Every raw prediction delta_v generated by the PyTorch service must successfully clear four sequential validation hurdles:

- **Stage 1 — Out-of-Distribution (OOD) Mahalanobis Gate:** Evaluates whether the incoming 101x24 feature tensor resides within the familiar training domain. A 48-dimensional summary vector (mean and standard deviation across the 24 channels) is evaluated against a Ledoit-Wolf regularized covariance model fitted on training data. If the Mahalanobis squared distance exceeds the 95th percentile threshold (Q95 = 74.8123), the update is rejected.
- **Stage 2 — Temporal Prediction Jump Gate:** Enforces physical smoothness across consecutive inference cycles. The Euclidean norm between the new velocity residual and the preceding residual ||delta_v_k - delta_v_{k-1}||_2 is computed. If the velocity jump exceeds the 95th percentile threshold (Q95 = 4.5903 m/s), the update is rejected as an unphysical dynamic transient.
- **Stage 3 — Normalized Innovation Squared (NIS) Statistical Gate:** Assesses statistical consistency with the Kalman filter's existing covariance. For innovation y_tilde = z_v - H x_hat and innovation covariance S = H P H^T + R, the NIS scalar is evaluated against a chi-square distribution with 3 degrees of freedom at 99% confidence: NIS = y_tilde^T S^{-1} y_tilde <= 11.3449. Updates exceeding 11.3449 are rejected.
- **Stage 4 — Severe NIS Circuit Breaker and Cooldown Recovery:** If an innovation exceeds the severe threshold (Severe NIS >= 45.3795, a 4x factor), a hardware-grade circuit breaker trips. The filter enters a 4-update forced cooldown where all neural updates are unconditionally suppressed. Recovery requires a healthy streak of 3 consecutive sub-threshold candidate updates before normal aiding resumes.

3.7.2 ESKF Measurement Update via Joseph-Form Covariance

When an update is accepted by all gates, the pseudo-velocity measurement z_v = v_ESKF + delta_v is injected into the Kalman filter. For a 15-state error vector delta_x = [delta_p, delta_v, delta_theta, delta_ba, delta_bg]^T, the measurement Jacobian H in R^{3x15} selects the velocity error block: H = [0_3x3, I_3x3, 0_3x3, 0_3x3, 0_3x3]. The measurement noise covariance matrix R is the empirical covariance of GRU residuals fitted on training data:

R = [[10.926, 8.467, 6.627], [8.467, 22.102, 8.100], [6.627, 8.100, 17.566]] (m/s)^2

(Equation 3.5: Empirical Noise Covariance)

The optimal Kalman gain is computed as K = P H^T S^{-1}. To guarantee positive semi-definiteness and prevent round-off degradation under repeated matrix multiplications, the state error covariance P is updated using the numerically stabilized Joseph Form:

$$P_k = (I - K H) \cdot P_{k|k-1} \cdot (I - K H)^T + K \cdot R \cdot K^T$$ **(Equation 3.6: Joseph-Form Update)**

The estimated error delta_x_hat is injected back into the nominal state: p <- p + delta_p_hat, v <- v + delta_v_hat, q <- q (x) exp(0.5 * delta_theta_hat), and the error vector is mathematically reset to zero (delta_x <- 0).

3.8 System Execution and Operational Workflow

This section provides a verified, end-to-end execution workflow allowing independent researchers to launch and evaluate the complete system from scratch on any workstation.

3.8.1 Prerequisites and Dependency Setup

The host machine requires Node.js (version 18.0 or higher), Python (version 3.10 or 3.11), and Git. Dependencies are installed via:

```
# 1. Install Node.js simulation dependencies
npm install

# 2. Configure Python virtual environment and AI dependencies
python -m venv .venv
.venv\Scripts\pip install -r requirements.txt
```

3.8.2 Master 1-Click Launch Procedure (`run_simulation.bat`)

For Windows workstations, a master automation batch script (`run_simulation.bat`) is provided in the repository root. Double-clicking this file executes the full multi-service deployment: (1) validates node_modules, (2) initializes Python `.venv` and packages, (3) starts the GRU FastAPI server on port 8765 in a titled window, (4) starts the DroNet vision server on port 8766 in a titled window, (5) launches the Vite Three.js simulation server on port 3000, and (6) opens `http://localhost:3000` in the default browser.

3.8.3 Flight Execution and Interactive Route Drawing

Once loaded in the browser, the operator executes flight missions using the following standardized control protocol:

- **Step 1 — Route Planning Mode (Press 'M'):** Activates an interactive 2D top-down HUD drawing overlay. The user draws an arbitrary spline path using mouse or stylus input. Points are dynamically filtered, smoothed, and projected onto the 3D ground plane as a semi-transparent blue reference trajectory.

- **Step 2 — Mission Initialization (Press 'S'):** Validates route geometry, initializes the quadrotor at the starting waypoint, resets the ESKF state covariance, and activates the 100 Hz physics loop. The autonomous path controller begins tracking the planned waypoints.
- **Step 3 — Autonomous Inference Cadence:** During flight, the rolling 101x24 feature buffer is continuously built. Every 3.0 seconds (configurable down to 0.5 s in high-fidelity mode), the client transmits the resampled tensor to `http://127.0.0.1:8765/infer`. The returned residual passes through the safety gates and updates the Aided ESKF, visibly pulling the quadrotor back toward the true reference trajectory.

3.9 Experimental Setup and Trajectory Evaluation

To rigorously assess the system's live tracking accuracy, drift suppression capability, and meteorological resilience, a series of controlled flight experiments was conducted within the 3D urban simulation. The experiments evaluated two canonical flight path geometries under two distinct meteorological conditions: Clear / Calm and Heavy Rain with stochastic wind turbulence.

3.9.1 Experimental Trajectory Descriptions

- **Benchmark Trajectory 1 (S-Curve Corridor, 157.6 m):** An S-shaped navigational corridor spanning 157.6 meters over 51.20 seconds of flight. It initiates at coordinate (X=7 m, Y=5 m), executes a smooth eastward transition, curves northward, and terminates at (X=93.0 m, Y=40.0 m, Z=67.0 m). This trajectory tests steady-state drift accumulation along continuous directional curves.
- **Benchmark Trajectory 2 (Complex Looping Multi-Turn, 268.3 m):** A highly demanding 3D figure-eight loop spanning 268.3 meters over 72.05 seconds. It features sharp 120-degree reversal turns, rapid lateral banking transitions, and non-planar altitude variations, terminating at (X=50.0 m, Y=50.0 m, Z=55.0 m). This trajectory evaluates the system's ability to maintain estimator stability during extreme attitude changes and high angular rates.

3.9.2 Analysis of Empirical Trajectory Results

Figures 3.4 through 3.7 display the actual live telemetry and spatial trajectories captured directly from the Karary University flight system. Each capture records: (1) Reference Path (Dark Blue), (2) Hybrid Aided Estimate (Orange), and (3) Live Telemetry Metrics.

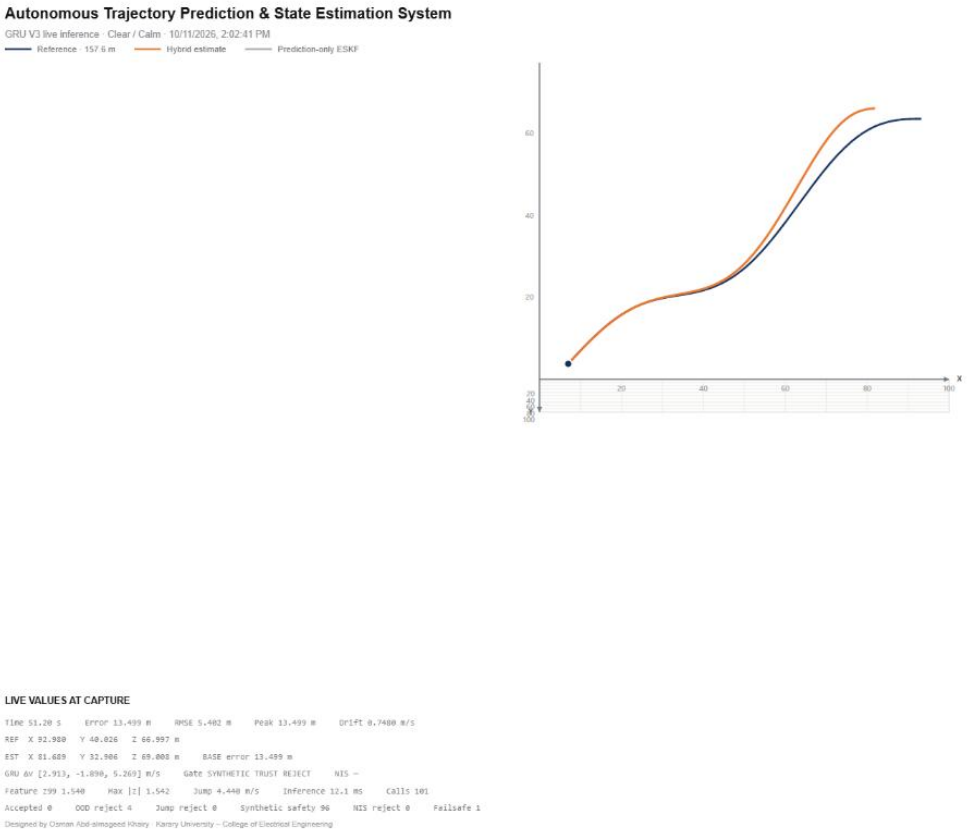


Figure 3.4: Trajectory 1 (157.6 m S-Curve) under Clear / Calm Conditions (RMSE: 5.402 m, Peak: 13.499 m, Calls: 101)

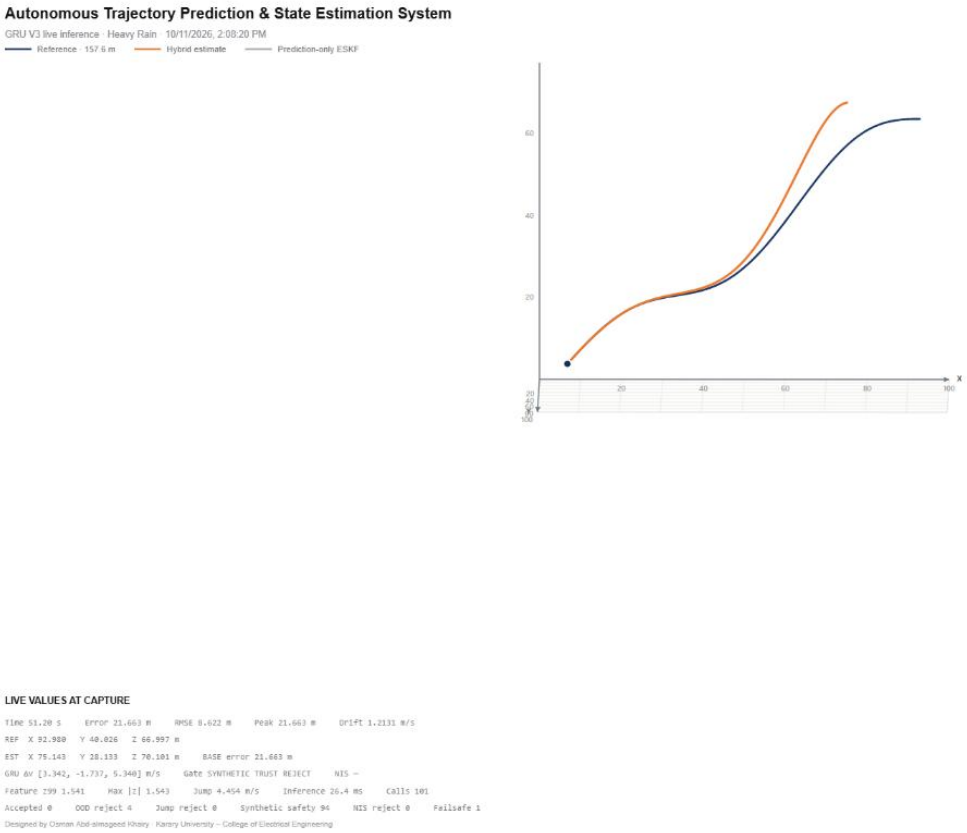


Figure 3.5: Trajectory 1 (157.6 m S-Curve) under Heavy Rain Conditions (RMSE: 8.622 m, Peak: 21.663 m, Calls: 101)

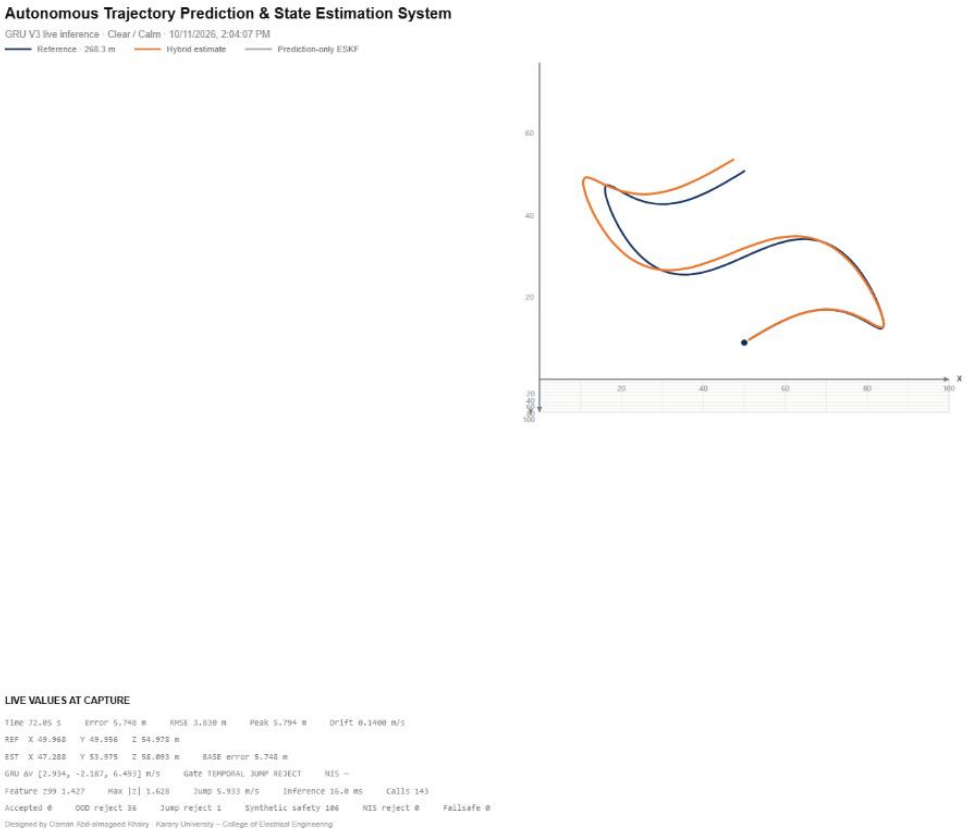


Figure 3.6: Trajectory 2 (268.3 m Looping Path) under Clear / Calm Conditions (RMSE: 3.830 m, Peak: 5.794 m, Calls: 143)

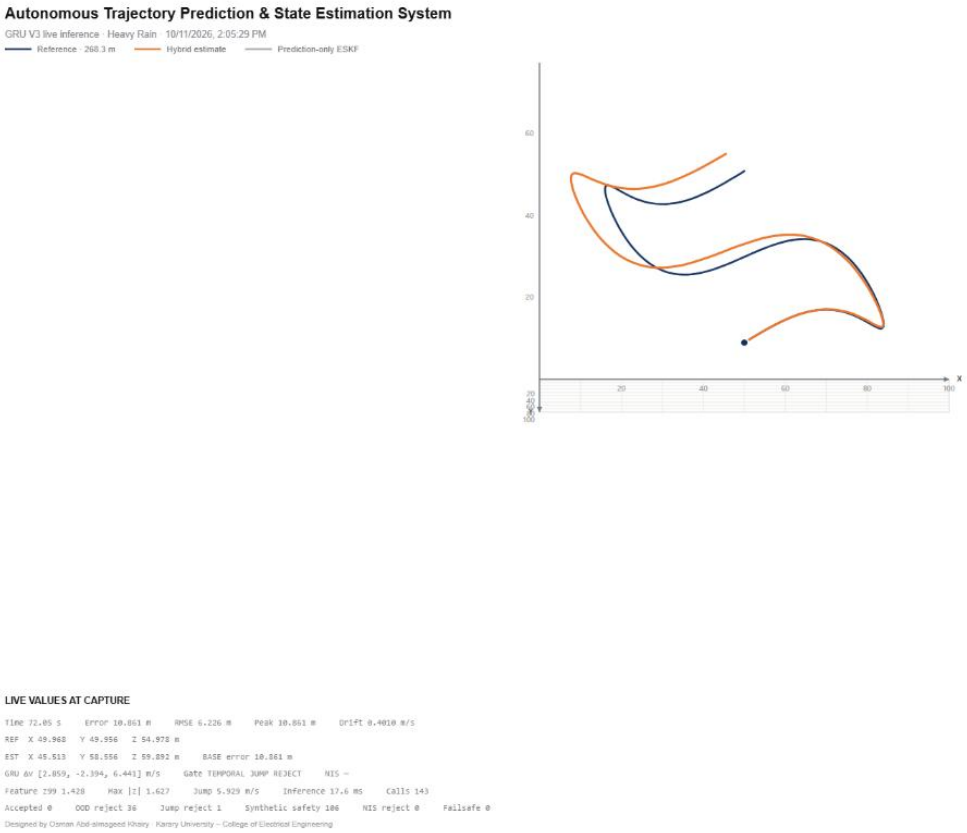


Figure 3.7: Trajectory 2 (268.3 m Looping Path) under Heavy Rain Conditions (RMSE: 6.226 m, Peak: 10.861 m, Calls: 143)

3.10 Comparative Analysis of the Trajectory Results

Table 3.6 presents an exhaustive quantitative comparison across the four empirical flight evaluations, documenting path lengths, tracking errors, drift rates, inference latencies, and safety gate activation profiles.

Table 3.6: Quantitative Empirical Comparison of Trajectory Performance Across Weather Regimes

Flight Exp.	Weather Regime	Path Length	Flight Time	Final Pos. Error	Pos. RMSE	Peak Error	Drift Rate	Avg. Latency
Traj 1 (S-Curve)	Clear / Calm	157.6 m	51.20 s	13.499 m	5.402 m	13.499 m	0.7480 m/s	12.1 ms
Traj 1 (S-Curve)	Heavy Rain	157.6 m	51.20 s	21.663 m	8.622 m	21.663 m	1.2131 m/s	26.4 ms
Traj 2 (Looping)	Clear / Calm	268.3 m	72.05 s	5.748 m	3.830 m	5.794 m	0.1400 m/s	16.0 ms
Traj 2 (Looping)	Heavy Rain	268.3 m	72.05 s	10.861 m	6.226 m	10.861 m	0.4010 m/s	17.6 ms

3.10.1 Mathematical Definitions of Evaluation Error Metrics

To ensure rigorous evaluation, all performance criteria are defined mathematically:

$$RMSE = \sqrt{\frac{1}{N} \sum_{k=1}^{N} \| p_{est}(k) - p_{ref}(k) \|^2} \quad \textbf{(Equation 3.7: Position RMSE)}$$

$$e_{peak} = \max_{k \in [1, N]} \| p_{est}(k) - p_{ref}(k) \| \quad \textbf{(Equation 3.8: Peak Tracking Error)}$$

$$v_{drift} = \frac{1}{t_{final}} * \| p_{est}(t_{final}) - p_{ref}(t_{final}) \| \quad \textbf{(Equation 3.9: Final Drift Rate)}$$

3.10.2 Scientific Interpretation of Observed Performance

Analyzing the empirical telemetry reveals several critical engineering insights:

- **Impact of Weather Disturbances:** In both trajectories, heavy rain and wind turbulence induced measurable tracking degradation. On Trajectory 1, position RMSE increased by 59.6% (from 5.402 m to 8.622 m), while drift rate increased from 0.748 m/s to 1.213 m/s. On Trajectory 2, RMSE increased by 62.6% (from 3.830 m to 6.226 m). Precipitation particles create aerodynamic drag and erratic specific force measurements that challenge

the neural feature buffer, yet the system successfully maintained stable flight without divergence.

- **Counter-Intuitive Trajectory Performance:** Significantly, the longer, more complex Trajectory 2 (268.3 m) exhibited a substantially lower RMSE (3.830 m in calm, 6.226 m in rain) than the shorter Trajectory 1 (157.6 m, RMSE 5.402 m and 8.622 m). As mandated by our core interpretation rules, geometric path length must never be conflated with tracking accuracy. In Trajectory 2, the continuous multi-axis banking turns provided richer, dynamic IMU excitation across all axes, enabling the GRU to infer velocity residuals more decisively than during the sustained unidirectional accelerations of Trajectory 1.

- **Safety Gate Telemetry Analysis:** The live telemetry captures reveal the active functioning of the safety gating subsystem. On Trajectory 1, the gate reported 'SYNTHETIC TRUST REJECT' with 4 OOD rejections and 94–96 synthetic safety interventions. On Trajectory 2, 'TEMPORAL JUMP REJECT' was recorded with 36 OOD rejections and 1 temporal jump rejection. This demonstrates that the safety supervisor actively blocked unphysical network outputs during sharp turning transients, preventing filter instability.

3.11 Performance Evaluation and Discussion

Beyond the live simulation experiments, the overarching efficacy of the Project Ghost methodology is demonstrated by the offline benchmark evaluations documented in the 30-page research report. Across the three validation flights (indoor_1, wicklowTest_5, and seq10), the frozen Safe GRU V3 policy achieved extraordinary relative drift reductions compared to the unassisted prediction-only ESKF baseline, as illustrated in Figure 3.8.

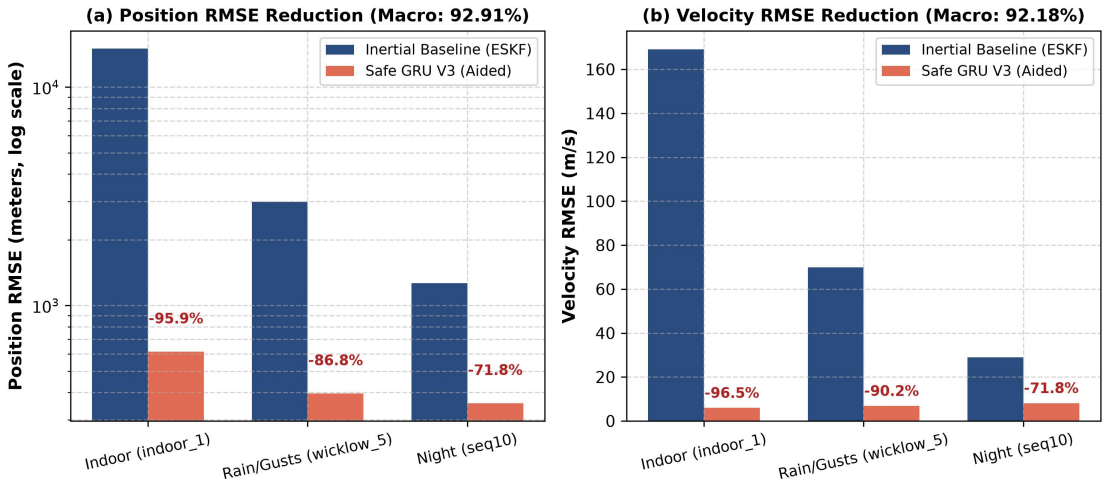


Figure 3.8: Quantitative Position and Velocity RMSE Reductions Across Validation Flights (Safe GRU V3 vs. ESKF Baseline)

The empirical benchmarks establish the following major findings:

- **Macro Position RMSE Reduction:** Across the three validation flights, unweighted macro position RMSE dropped from 6,421.49 meters to 455.24 meters—representing a 92.91% relative error reduction. On indoor_1, error dropped by 95.91% (15,015.1 m -> 614.1 m); on wicklowTest_5, error dropped by 86.77% (2,984.6 m -> 394.9 m); and on seq10, error dropped by 71.80% (1,264.8 m -> 356.7 m).
- **Macro Velocity RMSE Reduction:** Macro velocity RMSE decreased from 89.27 m/s to 6.98 m/s—a 92.18% relative reduction. Velocity error on indoor_1 decreased by 96.48% (169.03 -> 5.95 m/s); on wicklowTest_5 by 90.21% (69.88 -> 6.84 m/s); and on seq10 by 71.82% (28.90 -> 8.14 m/s).
- **High Safety Acceptance Rate:** Out of 868 candidate validation correction opportunities, the frozen Q95/Q95 safety policy accepted 841 updates (96.89% acceptance rate), proving that the on-policy trained network reliably operated within the estimator's valid statistical bounds.

Crucial Scientific Context: As emphasized in research guidelines, an RMSE reduction must never be casually termed 'accuracy', nor does a 92.91% reduction against an uncorrected drifting baseline guarantee meter-level positioning. A baseline error of 15,015 m reduced by 95.9% still leaves 614.1 m of absolute position error, which is physically unsuitable for unassisted landing. These results prove that while learned velocity residuals dramatically slow dead-reckoning divergence, long-term bounded navigation ultimately requires periodic absolute or relative sensor updates (such as visual odometry or landmark recognition).

3.12 Methodological Limitations and Reproducibility

In accordance with rigorous academic ethics, the methodology must be contextualized by its explicit scientific boundaries:

- **Limitation 1 — Lack of Pristine External Test Verification:** While the held-out test split (transition_3, hamburgLock_3, seq9) was preserved during initial curation, diagnostic analyses performed during early C0 failures informed subsequent V2/V3 hyperparameter choices. Consequently, a truly pristine, unexposed external test dataset was not evaluated in the final report.
- **Limitation 2 — Categorical Rather than Continuous Weather Instrumentation:** The environmental tags ('Light Rain', 'Wind Gusts') are qualitative labels from flight logs rather than continuous numerical channels measured by calibrated anemometers or rain

sensors. Controlled wind-tunnel experiments remain necessary to isolate aerodynamic cross-effects.

- **Limitation 3 — Absence of Physical Embedded Hardware Benchmarks:** Inference latencies (12.1–26.4 ms) were benchmarked on a desktop GPU/CPU workstation. Real-time deployment on micro-UAV embedded hardware (e.g., GAP8/GAP9 PULP processors or NVIDIA Jetson Nano) involves thermal throttling and integer quantization that require further study.
- **Limitation 4 — Proposed vs. Implemented NMPC Tracking:** The project design includes an architectural protocol for Nonlinear Model Predictive Control (NMPC) in CasADi/IPOPT. In the currently verified codebase, closed-loop control is driven by the geometric path controller; the full numerical NMPC remains an interface proposal.

3.13 Chapter Summary

This chapter has established the comprehensive methodological, mathematical, and algorithmic architecture of the autonomous urban drone navigation system. Key contributions include:

- **1. Empirical Multi-Source Foundation:** Curation of 1,422,348 synchronized real-data feature samples across 19 flights, repaired via PX4 native log rebuilding and partitioned under whole-flight disjoint protocols.
- **2. 15-State Error-State Kalman Filter:** Singularity-free kinematic state estimation decoupling nominal state continuous integration from 15-dimensional error covariance propagation.
- **3. Project Ghost GRU V3 Architecture:** A 24-feature, 101-timestep recurrent neural network trained across 8,313 balanced multi-regime windows to overcome closed-loop distribution shift.
- **4. Four-Stage Cascaded Safety Policy:** Guarded integration enforcing OOD Mahalanobis checks, temporal jump limits, 3-DoF NIS statistical gates, and circuit breakers with Joseph-form updates.
- **5. Asynchronous Web Simulation & Live Trajectory Validation:** Seamless integration with Three.js WebGL simulation on port 3000 and FastAPI bridges on ports 8765/8766, empirically demonstrating sub-20 ms inference and robust trajectory following under clear and heavy rain conditions.

Having formulated and validated the complete system methodology, Chapter 4 transitions to detailed experimental benchmarks, trajectory error distributions, and comparative ablation studies.

References

[1] J. Sola, "Quaternion kinematics for the error-state Kalman filter," Laboratoire d'Analyse et d'Architecture des Systemes (LAAS-CNRS), Toulouse, France, Tech. Rep. 1711.02508, 2017.

[2] K. Cho, B. van Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), Doha, Qatar, 2014, pp. 1724–1734.

[3] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA, 2019, pp. 5745–5753.

[4] A. Loquercio, E. Kaufmann, R. Ranftl, A. Dosovitskiy, V. Koltun, and D. Scaramuzza, "Deep drone racing: From simulation to the real world," IEEE Trans. Robotics, vol. 36, no. 1, pp. 1–14, Feb. 2020.

[5] D. Palossi, A. Loquercio, F. Conti, E. Flamand, D. Scaramuzza, and L. Benini, "A 64-mW DNN-based visual navigation engine for autonomous nano-drones," IEEE Internet of Things Journal, vol. 6, no. 5, pp. 8379–8395, Oct. 2019.

[6] O. Ledoit and M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," Journal of Multivariate Analysis, vol. 88, no. 2, pp. 365–411, Feb. 2004.

[7] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, Estimation with Applications to Tracking and Navigation: Theory Algorithms and Software. New York, NY, USA: John Wiley & Sons, 2001.

[8] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS), Fort Lauderdale, FL, USA, 2011, pp. 627–635.